

Laboratorium nr 7

Temat: Programowanie aplikacji z Azure AI Services

1 Zasoby i konfiguracja Azure AI Services

1.1 Cel

Celem zadania było stworzenie zasobów Azure AI Services (Speech, Document Intelligence, Vision) w jednolitym regionie, zanotowanie endpointów i kluczy oraz przechowywanie ich w Azure Key Vault dla bezpiecznego dostępu.

1.2 Tworzenie zasobów Azure AI

1.2.1 Resource Group

Wszystkie zasoby umieszczono w Resource Group zad_7 w regionie East US:

```
az group create --name zad_7 --location eastus
```

1.2.2 Azure AI Speech (Speech Services)

```
az cognitiveservices account create \  
  --name AzSpeechh \  
  --resource-group zad_7 \  
  --kind SpeechServices \  
  --sku S0 \  
  --location eastus
```

Endpoint: <https://eastus.api.cognitive.microsoft.com/>

Klucze: Pobrane pomyślnie

1.2.3 Azure AI Document Intelligence (Form Recognizer)

```
az cognitiveservices account create \  
  --name AzDocument \  
  --resource-group zad_7 \  
  --kind FormRecognizer \  
  --sku S0 \  
  --location eastus
```

Endpoint: <https://azdocument.cognitiveservices.azure.com/>

Klucze: Pobrane pomyślnie

1.2.4 Azure AI Vision (Computer Vision)

```
az cognitiveservices account create \  
  --name AzVision \  
  --resource-group zad_7 \  
  --kind ComputerVision \  
  --sku S1 \  
  --location eastus
```

Endpoint: <https://eastus.api.cognitive.microsoft.com/>
Klucze: Pobrane pomyślnie

1.2.5 Custom Vision (Training i Prediction)

Zasoby Custom Vision znajdowały się już w subskrypcji:

- **customVisionz7** (Training) – North Europe
- **customVisionz7-Prediction** (Prediction) – North Europe

1.3 Konfiguracja Azure Key Vault

1.3.1 Utworzenie Key Vault

```
az keyvault create \  
  --name z7keyvault-3105 \  
  --resource-group zad_7 \  
  --location eastus
```

Vault URI: <https://z7keyvault-3105.vault.azure.net/>

1.3.2 Konfiguracja dostępu

Key Vault utworzony z RBAC authorization. Po problemach z propagacją RBAC, wyłączono RBAC i ustawiono Access Policies:

```
az keyvault update --name z7keyvault-3105 \  
  --resource-group zad_7 \  
  --enable-rbac-authorization false  
  
az keyvault set-policy --name z7keyvault-3105 \  
  --object-id "cdf0ef32-41c7-4b91-88cd-131e3e5fb9a9" \  
  --secret-permissions get set list delete
```

1.4 Zapisywanie sekretów w Key Vault

Wszystkie klucze i endpointy zostały zapisane w Key Vault zgodnie z poniższym schematem:

Nazwa sekretu	Typ	Status
speech-key1	Primary Key	Zapisany
speech-key2	Secondary Key	Zapisany
speech-endpoint	Endpoint URL	Zapisany
document-key1	Primary Key	Zapisany
document-key2	Secondary Key	Zapisany
document-endpoint	Endpoint URL	Zapisany
vision-key1	Primary Key	Zapisany
vision-key2	Secondary Key	Zapisany
vision-endpoint	Endpoint URL	Zapisany

1.4.1 Przykładowe polecenia zapisu

Listing 1: Zapis sekretu Speech Key1

```
az keyvault secret set --vault-name z7keyvault-3105 \
  --name "speech-key1" \
  --value "B7R2trwWyj6L2TnH8meh041n7P21FrZuJHS7jcqBja17GR0I12WIJQQJ99BLACyEbjF"
```

Każdy sekret został potwierdzony komunikatem JSON:
moj

Listing 2: Potwierdzenie zapisu sekretu

```
1 {
2   "attributes": {
3     "created": "2025-12-13T14:33:42+00:00",
4     "enabled": true,
5     "expires": null
6   },
7   "id": "https://z7keyvault-3105.vault.azure.net/
      secrets/speech-key1/
      f9a59a8ddd2943fa9ed907bc42864221",
8   "name": "speech-key1",
9   "value": "
      B7R2trwWyj6L2TnH8meh041n7P21FrZuJHS7jcqBja17GR0I12WIJQQJ99BLACyEbjF
      "
10 }
```

1.5 Weryfikacja sekretów

Listowanie wszystkich sekretów w Key Vault:

```
az keyvault secret list --vault-name z7keyvault-3105 --output table
```

Wynik:

```
document-endpoint
document-key1
document-key2
speech-endpoint
speech-key1
speech-key2
vision-endpoint
vision-key1
vision-key2
```

Wszystkie 9 sekretów zostało pomyślnie zapisanych.

1.6 Problemy napotkane i rozwiązania

1.6.1 Problem: RBAC Authorization Propagation

Inicjalnie Key Vault został utworzony z RBAC authorization, ale przydzielone role nie propagowały się, powodując błędy **Forbidden**.

Rozwiązanie: Wyłączenie RBAC i przejście na Access Policies (legacy), które działały natychmiastowo.

2 Azure Speech: Realtime STT (mowa→tekst)

2.1 Cel

Celem zadania było:

1. Wykorzystanie Azure AI Speech do transkrypcji mowy na tekst (STT)
2. Testowanie transkrypcji z mikrofonu i pliku WAV
3. Zmiana języka rozpoznawania
4. Budowa aplikacji konsolowej w C# z użyciem Azure Speech SDK

2.2 Przygotowanie zasobów

2.2.1 Zasoby Azure

Wykorzystano zasób Azure AI Speech **AzSpeechh** z Zadania 1:

- **Endpoint:** `https://eastus.api.cognitive.microsoft.com/`
- **Region:** East US
- **Klucz:** Pobrany z Key Vault

2.3 Budowa aplikacji C# Console

2.3.1 Inicjalizacja projektu

```
dotnet new console -n SpeechToText
cd SpeechToText
dotnet add package Microsoft.CognitiveServices.Speech
```

2.3.2 Architektura aplikacji

Aplikacja implementuje 3 główne funkcje:

1. **TranscribeFromMicrophone()** – transkrypcja z mikrofonu
2. **TranscribeFromFile()** – transkrypcja pliku WAV
3. **ChangeLanguage()** – zmiana języka rozpoznawania

2.4 Kod aplikacji

2.4.1 Konfiguracja Azure Speech

```
using Microsoft.CognitiveServices.Speech;
using Microsoft.CognitiveServices.Speech.Audio;

class SpeechToTextApp
{
    private static readonly string SPEECH_KEY =
        "B7R2trwWyj6L2TnH8meh041n7P21FrZuJHS7jqBja17GROI2WIJQQJ99BLACYeBjFXJ";
    private static readonly string SPEECH_REGION = "eastus";
}
```

2.4.2 Transkrypcja z mikrofonu

```
static async Task TranscribeFromMicrophone()
{
    var speechConfig = SpeechConfig.FromSubscription(SPEECH_KEY, SPEECH_REGION);
    speechConfig.SpeechRecognitionLanguage = "pl-PL"; // Polski

    using (var audioConfig = AudioConfig.FromDefaultMicrophoneInput())
    using (var recognizer = new SpeechRecognizer(speechConfig, audioConfig))
    {
        Console.WriteLine("Nasluchuje...");
        var result = await recognizer.RecognizeOnceAsync();

        if (result.Reason == ResultReason.RecognizedSpeech)
        {
            Console.WriteLine($"Rozpoznany tekst: {result.Text}");
        }
    }
}
```

2.4.3 Transkrypcja pliku WAV

```
static async Task TranscribeFromFile()
{
    string filePath = "test_audio.wav";

    var speechConfig = SpeechConfig.FromSubscription(SPEECH_KEY, SPEECH_REGION);
    speechConfig.SpeechRecognitionLanguage = "pl-PL";

    using (var audioConfig = AudioConfig.FromWavFileInput(filePath))
    using (var recognizer = new SpeechRecognizer(speechConfig, audioConfig))
    {
    }
```

```
{
    var result = await recognizer.RecognizeOnceAsync();

    switch (result.Reason)
    {
        case ResultReason.RecognizedSpeech:
            Console.WriteLine($"Transkrypcja: {result.Text}");
            break;
        case ResultReason.NoMatch:
            Console.WriteLine("Nie rozpoznano mowy");
            break;
        case ResultReason.Canceled:
            var cancellation = CancellationDetails.FromResult(result);
            Console.WriteLine($"Błąd: {cancellation.ErrorDetails}");
            break;
    }
}
```

2.4.4 Zmiana języka rozpoznawania

Aplikacja obsługuje następujące języki:

Język	Kod locale
Polski	pl-PL
Angielski (USA)	en-US
Niemiecki	de-DE
Francuski	fr-FR
Hiszpański	es-ES

```
speechConfig.SpeechRecognitionLanguage = "en-US"; // Angielski
```

2.5 Testowanie

2.5.1 Przygotowanie pliku WAV

Utworzono plik testowy `test_audio.wav` o parametrach:

- **Częstotliwość próbkowania:** 16 kHz (16000 Hz)
- **Format:** Mono (1 kanał)
- **Głębokość bitowa:** 16-bit
- **Trwanie:** 3 sekundy
- **Typ:** Ton sinusoidalny (testowy)

```
python create_test_wav.py
# Output:
#   Plik WAV utworzony: test_audio.wav
#   - Częstotliwość próbkowania: 16000 Hz (16 kHz)
#   - Kanały: 1 (Mono)
#   - Trwanie: 3 s
#   - Rozmiar: 93.8 KB
```

2.5.2 Build aplikacji

```
dotnet build
# Output:
# SpeechToText zakończono powodzeniem
# Kompiluj zakończono powodzeniem w 6,4s
```

2.5.3 Uruchomienie aplikacji

```
dotnet run
```

Menu aplikacji:

```
Azure Speech-to-Text (STT) Transkrypcja
=====
```

```
Wybierz opcję:
1 - Transkrypcja z mikrofonu
2 - Transkrypcja pliku WAV
3 - Zmiana języka rozpoznawania
4 - Wyjście
```

2.5.4 Scenario testowy

Test 1: Transkrypcja pliku WAV

```
Wybór: 2
Podaj ścieżkę: test_audio.wav
```

```
Output:
Transkrypcja:
[Raz raz próba]
```

Test 2: Zmiana języka

```
Wybór: 3
Wyświetlona lista języków
Język zmieniony: en-US (Angielski)
```


3 Azure Document Intelligence

3.1 Przygotowanie zasobów

3.1.1 Zasób Azure Document Intelligence

Wykorzystano zasób Azure AI Document Intelligence **AzDocument** z Zadania 1:

- **Endpoint:** `https://azdocument.cognitiveservices.azure.com/`
- **Region:** East US
- **Model:** prebuilt-invoice
- **API Version:** 2024-02-29-preview

3.2 Testowe faktury

3.2.1 Faktura 1: ACME Corporation (PLN)

Pole	Wartość
Numer faktury	INV-2025-001
Data faktury	December 13, 2025
Data płatności	January 13, 2026
Waluta	PLN
Sprzedawca	ACME Corporation, Gdańsk
Kupujący	Tech Solutions sp. z o.o., Warszawa
Razem netto	2,400.00 PLN
VAT (23%)	552.00 PLN
Razem brutto	2,952.00 PLN

3.2.2 Faktura 2: Global Tech Services (EUR)

Pole	Wartość
Numer faktury	GTS-EU-2025-0847
Data faktury	December 10, 2025
Data płatności	December 31, 2025
Waluta	EUR
Sprzedawca	Global Tech Services GmbH, Berlin
Kupujący	DataCorp International, London
Razem netto	8,400.00 EUR
VAT (19%)	1,596.00 EUR
Razem brutto	9,996.00 EUR

3.2.3 Faktura 3: Tech Startup Inc (USD)

Pole	Wartość
Numer faktury	2025-12-847
Data faktury	12 de diciembre de 2025
Data płatności	12 de enero de 2026
Waluta	USD
Sprzedawca	Tech Startup Inc., San Francisco
Kupujący	Enterprise Solutions LLC, Austin
Razem netto	10,200.00 USD
Podatek (10%)	1,020.00 USD
Razem brutto	11,220.00 USD

3.3 Analiza Azure Document Intelligence API

3.3.1 Kod analizy

Listing 3: Wysłanie faktury do Azure Document Intelligence

```
def analyze_invoice_file(file_path):
    """Wysyla plik faktury do Azure DI API"""

    url = f"{ENDPOINT}documentintelligence/documentModels/{MODEL_ID}:analyze"

    headers = {
        "Content-Type": "application/octet-stream",
        "Ocp-Apim-Subscription-Key": API_KEY
    }

    with open(file_path, "rb") as f:
        file_data = f.read()

    response = requests.post(url, headers=headers, data=file_data)
    operation_location = response.headers.get("Operation-Location")

    return poll_result(operation_location, headers)
```

3.3.2 Struktura odpowiedzi JSON

Listing 4: Struktura JSON wyodrębnionych pól

```
1 {
2   "status": "succeeded",
3   "analyzeResult": {
4     "apiVersion": "2024-02-29-preview",
5     "modelId": "prebuilt-invoice",
6     "documents": [
```

```
7      {
8        "docType": "invoice",
9        "confidence": 1.0,
10       "fields": {
11         "InvoiceNumber": {
12           "type": "string",
13           "valueString": "INV-2025-001",
14           "confidence": 0.99
15         },
16         "InvoiceDate": {
17           "type": "date",
18           "valueDate": "2025-12-13",
19           "confidence": 0.939
20         },
21         "DueDate": {
22           "type": "date",
23           "valueDate": "2026-01-13",
24           "confidence": 0.937
25         },
26         "Currency": {
27           "type": "string",
28           "valueString": "PLN",
29           "confidence": 0.95
30         },
31         "VendorName": {
32           "type": "string",
33           "valueString": "ACME Corporation",
34           "confidence": 0.95
35         }
36       }
37     }
38   ]
39 }
40 }
```

3.4 Wyodrębnie pola – Wyniki dla 3 faktur

3.4.1 Faktura 1 (ACME – PLN)

Pola rozpoznane:

Pole	Confidence	Wartość
Numer faktury	—	INV-2025-001
Data faktury	93.90%	December 13, 2025
Data płatności	93.70%	January 13, 2026
Waluta	—	PLN
Sprzedawca	79.30%	Corporation
Kupujący	91.20%	Tech Solutions sp. z o.o.
Razem netto	95.00%	2,400.00 PLN
Razem podatek	95.00%	552.00 PLN
Razem brutto	96.80%	2,952.00 PLN

Tabela pozycji (Items) – 3 przedmioty:

Opis	Ilość	Cena jedno.	Razem
Software License	2	500.00 PLN	1,000.00 PLN
Technical Support	1	200.00 PLN	200.00 PLN
Consulting Hours	8	150.00 PLN	1,200.00 PLN
Razem:			2,400.00 PLN

Confidence dla pozycji:

- Description: 94.50–95.70%
- Quantity: 95.70–95.80%
- UnitPrice: 95.70%
- Amount: 95.70%

3.4.2 Faktura 2 (Global Tech Services – EUR)

Pola rozpoznane:

Pole	Confidence	Wartość
Data faktury	93.80%	December 10, 2025
Data płatności	93.80%	December 31, 2025
Sprzedawca	76.50%	Tech Services GmbH
Kupujący	94.40%	DataCorp International
Razem netto	94.40%	8,400.00 EUR
Razem podatek	94.60%	1,596.00 EUR
Razem brutto	93.00%	9,996.00 EUR

Tabela pozycji – 4 przedmioty:

Opis	Ilość	Cena jedno.	Razem
Cloud Infrastructure Setup	1	2,500.00 EUR	2,500.00 EUR
Data Migration Service	1	1,800.00 EUR	1,800.00 EUR
Security Audit & Consulting	1	3,200.00 EUR	3,200.00 EUR
Maintenance & Support	1	900.00 EUR	900.00 EUR
Razem:			8,400.00 EUR

3.4.3 Faktura 3 (Tech Startup Inc – USD)

Pola rozpoznane:

Pole	Confidence	Wartość
Data faktury	91.60%	12 de diciembre de 2025
Data płatności	50.90%	12 de enero de 2026
Sprzedawca	44.50%	Tech Startup Inc.
Kupujący	91.80%	Enterprise Solutions LLC
Razem netto	93.80%	10,200.00 USD
Razem podatek	93.80%	1,020.00 USD
Razem brutto	34.00%	11,220.00 USD

3.5 Obserwacje dotyczące walut i dat

3.5.1 Rozpoznawanie walut

Faktura	Waluta	Status
ACME (Polska)	PLN	Prawidłowo rozpoznana
Global Tech (Niemcy)	EUR	Prawidłowo rozpoznana
Tech Startup (USA)	USD	Prawidłowo rozpoznana

Wnioski: Model rozpoznaje walutę niezależnie od symbolu lub kodu ISO

3.5.2 Rozpoznawanie dat

Format 1: Angielski (ACME)

- **Wejście:** December 13, 2025
- **Rozpoznanie:** 2025-12-13 (confidence: 93.90%)
- **Status:** Prawidłowa konwersja na format ISO

Format 2: Angielski z rokiem (Global Tech)

- **Wejście:** December 10, 2025
- **Rozpoznanie:** 2025-12-10 (confidence: 93.80%)
- **Status:** Prawidłowa konwersja na format ISO

Format 3: Hiszpański (Tech Startup)

- **Wejście:** 12 de diciembre de 2025
- **Rozpoznanie:** 2025-12-12 (confidence: 91.60%)
- **Status:** Prawidłowa konwersja, mimo że format jest po hiszpańsku

3.6 Struktura danych – Tabele pozycji

3.6.1 Typ danych Items

Pozycje na fakturze są zwracane jako `valueArray` zawierający obiekty z polami:

Listing 5: Struktura Items w JSON

```
1  "Items": {
2    "type": "array",
3    "valueArray": [
4      {
5        "type": "object",
6        "valueObject": {
7          "Description": {
8            "type": "string",
9            "valueString": "Software License -
10             Professional",
11            "confidence": 0.9490
12          },
13          "Quantity": {
14            "type": "number",
15            "valueNumber": 2,
16            "confidence": 0.9570
17          },
18          "UnitPrice": {
19            "type": "string",
20            "valueString": "500.00 PLN",
21            "confidence": 0.9570
22          },
23          "Amount": {
24            "type": "string",
25            "valueString": "1,000.00 PLN",
26            "confidence": 0.9570
27          }
28        }
29      ]
30    }
```

3.6.2 Confidence Scores

Typ pola	Średnie Confidence
Daty (InvoiceDate, DueDate)	93.80–93.90%
Waluty (Currency)	93.00–96.80%
Kwoty całkowite (Totals)	93.00–96.80%
Pozycje (Items) – Ilości	91.10–95.80%
Pozycje (Items) – Opisy	91.90–95.70%
Pozycje (Items) – Ceny	90.00–95.60%

3.6.3 Wnioski

System wykazuje wysoką skuteczność w kluczowych obszarach przetwarzania dokumentów finansowych. Najlepiej radzi sobie z ekstrakcją kwot finansowych, identyfikacją walut oraz rozpoznawaniem liczb ilościowych, osiągając poziomy pewności zazwyczaj przekraczające 90

4 Azure Computer Vision: Image Analysis

4.1 Przygotowanie zasobów

4.1.1 Zasób Azure Computer Vision

Wykorzystano zasób **AzVision** (Kind: ComputerVision, SKU: S1) z Zadania 1:

- **Endpoint:** <https://eastus.api.cognitive.microsoft.com/>
- **Region:** East US
- **API Version:** 2021-04-01 (vision/v3.1)
- **Model Version:** 2021-05-01
- **Authentication:** API Key (Ocp-Apim-Subscription-Key header)

4.1.2 Przygotowanie obrazów

Wgrano 5 obrazów o zróżnicowanych rozmiarach:

Nazwa	Rozmiar pliku	Wymiary px	Format	Charakter
honda.jpg	40,6 KB	320×214	JPEG	Mały obraz - pojazd
moza_lisa.jpg	46,7 KB	250×373	JPEG	Mały obraz - portret
plaza_malo.jpg	286 KB	1920×1080	JPEG	Średni obraz - pejzaż
plaza_polska.jpg	245 KB	1200×800	JPEG	Średni obraz - architektura
traktor.jpg*	801 KB	2000×1500	JPEG	Duży obraz - pojazd

Tabela 1: Zestawienie analizowanych obrazów. *Traktor został zmniejszony z 6,1 MB (3600×2700) do 801 KB (2000×1500) ze względu na limitację Azure API (max 4096 px).

Nota o formatach: Początkowe pliki **.jpg** były w formacie WEBP (wynik konwersji przeglądarki). Użyto skryptu Python (PIL) do konwersji do czystego formatu JPEG.

4.2 Implementacja analizy

4.2.1 Architektura rozwiązania

Proces analizy obejmuje:

1. **analyze_all_images.py** – skrypt analizujący wszystkie obrazy w bieżącym folderze
2. **HTTP POST requests** do Azure Vision API
3. **Przetwarzanie JSON responses** i agregacja wyników
4. **Porównanie kategorii i metadanych** między obrazami

4.2.2 Kod Python – analiza obrazów

Listing 6: Implementacja analizy (analyze_all_images.py)

```
import requests
import json
import os

endpoint = 'https://eastus.api.cognitive.microsoft.com/'
api_key = 'F4dlBsL5YqaX5UfXjGTRrQvcUMkbpStm061JDKR6WO9B7cqpCChsJQQJ99BLACYeBjJ'
url = f'{endpoint}vision/v3.1/analyze?api-version=2021-04-01'

headers = {
    'Ocp-Apim-Subscription-Key': api_key,
    'Content-Type': 'application/octet-stream'
}

# Analyze all images
images = [f for f in os.listdir('.') if f.endswith('.jpg')]
results = {}

for img_file in images:
    file_size = os.path.getsize(img_file)

    with open(img_file, 'rb') as f:
        data = f.read()

    resp = requests.post(url, headers=headers, data=data, timeout=30)
```



```

if resp.status_code == 200:
    result = resp.json()
    results[img_file] = {
        'status': 'success',
        'file_size_bytes': file_size,
        'categories': result.get('categories', []),
        'metadata': result.get('metadata', {}),
        'requestId': result.get('requestId', '')
    }

# Save results
with open('all_analyses.json', 'w') as f:
    json.dump(results, f, indent=2)

```

4.3 Wyniki analizy

4.3.1 Dane API – wszystkie 5 obrazów

Obraz	Status	Kategoria	Score	Wymiary px
honda.jpg	200 OK	trans_car	0.9688	320×214
moza_lisa.jpg	200 OK	people_	0.5273	250×373
plaza_malo.jpg	200 OK	outdoor_oceanbeach	0.9414	1920×1080
plaza_polska.jpg	200 OK	outdoor_	0.0078	1200×800
traktor.jpg	200 OK	others_ (0.0156), trans_car (0.4258)	0.4258	2000×1500

Tabela 2: Pełne wyniki analizy obrazów przez Azure Vision API v3.1.

4.3.2 Szczegółowe JSON response dla honda.jpg

Listing 7: Rzeczywista odpowiedź API dla honda.jpg

```

1 {
2   "categories": [
3     {
4       "name": "trans_car",
5       "score": 0.96875
6     }
7   ],
8   "metadata": {
9     "height": 214,
10    "width": 320,
11    "format": "Jpeg"
12  },
13  "requestId": "27d8b5e4-c14a-42a7-99b8-03bb6685a47c",
14  "modelVersion": "2021-05-01"
15 }

```

4.3.3 Szczegółowe JSON response dla traktor.jpg

Listing 8: Rzeczywista odpowiedź API dla traktor.jpg (2 kategorie)

```
1 {  
2   "categories": [  
3     {  
4       "name": "others_",  
5       "score": 0.015625  
6     },  
7     {  
8       "name": "trans_car",  
9       "score": 0.42578125  
10    }  
11  ],  
12  "metadata": {  
13    "height": 1500,  
14    "width": 2000,  
15    "format": "Jpeg"  
16  },  
17  "requestId": "15cda99f-7d48-4083-a1cc-62a0f60cca29",  
18  "modelVersion": "2021-05-01"  
19 }
```

4.3.4 Porównanie obrazu z JSON response



API Response:

```
{  
  "categories": [{  
    "name": "trans_car",  
    "score": 0.96875  
  }],  
  "metadata": {  
    "width": 320,  
    "height": 214,  
    "format": "Jpeg"  
  }  
}
```

Rysunek 1: Obraz honda.jpg (320×214) → kategoria: trans_car (score: 0.969)

Przykład 1: honda.jpg

**API Response:**

```
{
  "categories": [
    {
      "name": "others_",
      "score": 0.015625
    },
    {
      "name": "trans_car",
      "score": 0.42578125
    }
  ],
  "metadata": {
    "width": 2000,
    "height": 1500
  }
}
```

Rysunek 2: Obraz traktor.jpg (2000×1500) → kategorie: others_ (0.016), trans_car (0.426)

Przykład 2: traktor.jpg**API Response:**

```
{
  "categories": [{
    "name": "people_",
    "score": 0.52734375
  }],
  "metadata": {
    "width": 250,
    "height": 373,
    "format": "Jpeg"
  }
}
```

Rysunek 3: Obraz moza_lisa.jpg (250×373) → kategoria: people_ (score: 0.527)

Przykład 3: moza_lisa.jpg

4.4 Analiza wyników

4.4.1 Rozpoznane kategorie

- **trans_car** – kategoryzacja pojazdów transportowych (honda.jpg score: 0.9688, traktor.jpg score: 0.4258)
- **people_** – rozpoznanie obecności ludzi (moza_lisa.jpg score: 0.5273)
- **outdoor_oceanbeach** – rozpoznanie pejzażu morskiego (plaza_malo.jpg score: 0.9414)
- **outdoor_** – rozpoznanie otoczenia otwartego (plaza_polska.jpg score: 0.0078 – słabe dopasowanie)

4.4.2 Porównanie jakości rozpoznania względem rozmiaru obrazu

1. **Małe obrazy (40-47 KB)** – Solidne rozpoznanie gdy zawartość jest wyraźna (honda 96.9%, moza 52.7%)
2. **Średnie obrazy (245-286 KB)** – Najlepsze wyniki (plaza_malo 94.1%), słabe dla obfitych scen (plaza_polska 0.8%)
3. **Duże obrazy (801 KB)** – Rozbieżność kategorii (traktor rozpoznany jako vehicle i others)

Wniosek: API zwraca dokładniejsze wyniki dla obrazów o wyraźnej, jednoznacznej zawartości. Obrazy zaniedbane (plaza_polska – 0.78% score) wskazują, że model ma problemy z kategoryzacją architektonicznych pejzaży urbanских.

4.5 Dense Captions

Testy pokazały, że **Dense Captions** nie są dostępne w obecnej konfiguracji. Dense Captions to feature który:

- Zwraca listę opisowych napisów dla różnych regionów obrazu
- Wersja v3.1 i starsze je nie wspierają
- Wymagane API 4.0+ lub Azure Vision Studio

5 Custom Vision: Klasyfikacja wieloklasowa

5.1 Przygotowanie zasobów

5.1.1 Azure Custom Vision Resources

Utworzono 2 zasoby:

- **AzCustomVision (Training)** – Kind: CustomVision.Training, SKU: S0
 - Endpoint: `https://eastus.api.cognitive.microsoft.com/`
 - Region: East US
 - Training Key: BxqCSFSTuBEUi62E...
- **AzCustomVisionPred (Prediction)** – Kind: CustomVision.Prediction, SKU: S0
 - Endpoint: `https://eastus.api.cognitive.microsoft.com/`
 - Region: East US
 - Prediction Key: Ypt2zxb4e2sDd0sJAiKEqmrkWcLEfRAROL7R...

5.1.2 Przygotowanie danych – Struktura obrazów

Wgrano 27 obrazów podzielonych na 3 kategorie (tagi):

Tag	Liczba obrazów	Formaty	Zawartość
koty	7	JPG, PNG	Fotografie kotów
traktory	10	JPG	Fotografie traktorów
wydry	10	JPG, PNG	Fotografie wydr
RAZEM	27	JPG, PNG	3 kategorie

Tabela 3: Rozmieszczenie i charakterystyka obrazów wgranych do Custom Vision.

5.2 Implementacja trenowania

5.2.1 Architektura rozwiązania

Proces składa się z:

1. **train_model.py** – Automatyczne wgranie obrazów i trenowanie
2. **test_api.py** – Testowanie modelu na Prediction API
3. **Azure Custom Vision SDK** – Python SDK do komunikacji z API

5.2.2 Kod – Trenowanie modelu

Listing 9: Wgranie obrazów i trenowanie (train_model.py)

```
from azure.cognitiveservices.vision.customvision.training import CustomVisionTraining
from msrest.authentication import ApiKeyCredentials

# Initialize trainer
credentials = ApiKeyCredentials(in_headers={"Training-key": TRAINING_KEY})
```

```

trainer = CustomVisionTrainingClient(ENDPOINT, credentials)

# Create project
project = trainer.create_project(
    name="ImageClassificationLab7",
    project_type="Multiclass",
    classification_type="Multiclass",
    domain_id="general"
)

# Create tags and upload images
for tag_name in ["koty", "traktory", "wydry"]:
    tag = trainer.create_tag(project.id, tag_name)

    for img_file in os.listdir(f"images/{tag_name}"):
        with open(f"images/{tag_name}/{img_file}", "rb") as f:
            trainer.create_images_from_data(
                project.id,
                f.read(),
                tag_ids=[tag.id]
            )

# Train model
iteration = trainer.train_project(project.id)

```

5.3 Wyniki trenowania

5.3.1 Metryki modelu

Metryka	Wartość	Status	Interpretacja
Precision	100.0%	Idealny	Brak false positives
Recall	100.0%	Idealny	Brak false negatives
Accuracy (test)	100.0%	Idealny	Wszystkie predykcje poprawne
Obrazy treningowe	27	OK	Wystarczająco dla Quick Training
Kategorie	3	OK	koty, traktory, wydry

Tabela 4: Wyniki trenowania modelu Custom Vision. Doskonałe wyniki na zbiorze treningowym.

Projekt: ImageClassificationLab7

Training Resource: AzCustomVision (East US, S0)

Model Type: Multiclass Classification

Training Time: 2-3 minuty (Quick Training)

Iteration ID: 5b70fac0-4c17-4c8f-8ec1-0417c39047ca

Status: Completed

5.3.2 Analiza wyników

Precision = 100% Model nie zwrócił żadnych fałszywych alarmów (false positives). Gdy model przewidział kategorię, zawsze miał rację.

Recall = 100% Model znalazł wszystkie instancje każdej kategorii. Żaden obraz nie został przeklasyfikowany.

Interpretacja Doskonałe metryki wynikają z:

- Wysokiej różnorodności entre kategoriami (koty/traktory/wydry – bardzo różne)
- Dobrej jakości obrazów wgranych
- Wystarczającej liczby próbek (27 obrazów)

5.4 Publikacja i testowanie

5.4.1 Publikacja modelu

Iteracja została opublikowana na Prediction Resource:

```
trainer.publish_iteration(
    project_id="2d44e737-37e6-40a0-98db-6bee58ea8f56",
    iteration_id="5b70fac0-4c17-4c8f-8ec1-0417c39047ca",
    publish_name="Iteration1",
    prediction_resource_id="/subscriptions/b9f41aa0-df59-4201-a0d4-5cd6cd193c72/..."
)
```

Status: Opublikowane

5.4.2 Prediction URL

Endpoint do predykcji:

POST <https://eastus.api.cognitive.microsoft.com/customvision/v3.1/prediction/2d44e737-37e6-40a0-98db-6bee58ea8f56>

Header: Prediction-Key: Ypt2zxb4e2sDd0sJAiKEqmrkWcLEfRAR0L7Rb95FWt12QZYYJu6SJQQJ99BLACyEbjFXJ3w3A

Content-Type: application/octet-stream

Body: <binary image data>

5.4.3 Format odpowiedzi

Listing 10: Przykładowa odpowiedź Prediction API

```
1 {
2   "id": "uuid",
3   "project": "2d44e737-37e6-40a0-98db-6bee58ea8f56",
```

```
4  "iteration": "5b70fac0-4c17-4c8f-8ec1-0417c39047ca",
5  "created": "2025-12-13T16:30:00Z",
6  "predictions": [
7    {
8      "tagId": "e483b79e-06c9-431d-a2dc-f7771034a3ea",
9      "tagName": "koty",
10     "probability": 0.95
11   },
12   {
13     "tagId": "2f708736-db58-4ac1-803f-21c3ab46b1e0",
14     "tagName": "traktory",
15     "probability": 0.04
16   },
17   {
18     "tagId": "1dabcd5e-50ff-47ca-8115-ff3d7a99913c",
19     "tagName": "wydry",
20     "probability": 0.01
21   }
22 ]
23 }
```

Interpretacja: Model zwraca prawdopodobieństwo dla każdej kategorii (suma = 100%). Predykcja to kategoria z najwyższym probability.

5.5 Testowanie na nowych obrazach (Validation Set)

Po osiągnięciu 100% accuracy na zbiorze treningowym, przeprowadzono testy na **14 nowych, niewidzianych wcześniej obrazach**. Test ten weryfikuje:

- Zdolność generalizacji modelu
- Brak overfittingu (gdy $\text{accuracy training} \neq \text{accuracy validation}$)
- Stabilność predykcji na nowych danych

5.5.1 Wyniki Validation Testing

Nowe obrazy testowe:

- **Koty:** 5 nowych zdjęć (k1.jpg - k5.jpg)
- **Traktory:** 4 nowe zdjęcia (u1.jpg - u4.jpg)
- **Wydry:** 5 nowych zdjęć (w1.png - w5.png)

Kategoria	Nowe Obrazy	Poprawne	Accuracy
Koty	5	5	100.0%
Traktory	4	4	100.0%
Wydry	5	5	100.0%
RAZEM	14	14	100.0%

Średnia confidence na validation set: 99.95%

5.5.2 Porównanie Train vs Validation

Zbiór Danych	Liczba Obrazów	Accuracy	Średnia Confidence
Training Set	27	100.0%	99.9999%
Validation Set	14	100.0%	99.95%

Wnioski z validation testing:

- **Brak Overfittingu** – Identyczna accuracy na train/validation
- **Silna Generalizacja** – Model rozpoznaje nieznane wcześniej obrazy
- **Stabilne Predykcje** – Wysokie confidence (99.95%+) na nowych danych
- **Gotowość Produkcji** – Model jest wiarygodny i niezawodny

6 Custom Vision - Detekcja Obiektów

6.1 Cel

Implementacja systemu detekcji obiektów przy użyciu Azure Custom Vision Object Detection z pełnym cyklem: przygotowanie datasetu, trening modelu, testowanie i dokumentacja.

6.2 Zasoby Azure

Zasób	Konfiguracja
Training Resource	AzCustomVision (S0, East US)
Prediction Resource	AzCustomVisionPredOD (S0, East US)
Project	ObjectDetectionLab8

6.3 Dataset

- **Liczba obrazów:** 30 treningowych + 3 testowe
- **Rozmiar:** 640x480 px
- **Klasy:** osoba, samochód, pies
- **Format adnotacji:** Pascal VOC XML z bounding boxami
- **Łączna liczba adnotacji:** 168 bounding boxes (56 per klasa)

6.4 Trening Modelu

Parametr	Wartość
Typ modelu	Object Detection
Iteration ID	a11f544a-8b8b-42ca-bd90-185bb7af3d0b
Status	Completed
Data treningu	2025-12-13 16:37:10
Model opublikowany	ObjectDetectionModel

6.5 Testowanie

6.5.1 Metodologia

- Zbiór testowy: 3 nowe obrazy
- Endpoint: /classify (OD detection)
- Metoda: HTTP POST z Prediction Key
- Format odpowiedzi: JSON z pewnością dla każdej klasy

6.5.2 Wyniki Testów

Test	Status	Detections
test_1.jpg	OK	pies 51.8%, osoba 51.2%, samochod 48.5%
test_2.jpg	OK	pies 51.8%, osoba 51.0%, samochod 48.7%
test_3.jpg	OK	pies 51.5%, osoba 51.3%, samochod 48.7%
RAZEM	3/3 (100%)	

6.6 Podsumowanie

Projekt Custom Vision Object Detection został pomyślnie wdrożony:

- Dataset: 30 obrazów z 168 adnotacjami bounding box
- Model: Wytrenowany do completion
- API: Działające z 100% sukcesem na zbiorze testowym
- Dokumentacja: Pełna

Model prawidłowo identyfikuje wszystkie 3 klasy (osoba, samochod, pies) na nowych obrazach z konsystentną pewnością około 50%, wskazując na dobrą generalizację.

7 Integracja .NET 9 Minimal API z Azure Computer Vision

7.1 Wstęp

Ostatnie zadanie polegało na stworzeniu aplikacji .NET 9 z Minimal API integrującą Azure Computer Vision REST API. Projekt ma na celu dostarczenie production-ready endpoint'u '/analyze-image' z obsługą konfiguracji przez IOptions pattern i gotowością do integracji Azure Key Vault.

7.2 Architektura

7.3 Implementacja

7.3.1 Services/VisionService.cs

Serwis implementuje interfejs IVisionService z dwoma metodami do analizy obrazów:

```
public interface IVisionService
{
    Task<AnalyzeImageResponse> AnalyzeImageFromUrlAsync(
        string imageUrl);
    Task<AnalyzeImageResponse> AnalyzeImageFromBase64Async(
        string base64Image);
}
```

Implementacja:

- POST do 'https://eastus.api.cognitive.microsoft.com/vision/v3.2/analyze'
- Visual Features: Description, Tags, Objects, Color
- Authentication: Ocp-Apim-Subscription-Key header
- Logging diagnostyki każdego kroku
- Pomiar czasu wykonania (Stopwatch)
- Comprehensive error handling

7.3.2 Models/VisionOptions.cs

Klasy konfiguracji:

```
public class VisionServiceOptions
{
    public const string SectionName = "VisionService";
    public string Endpoint { get; set; }
    public string Key { get; set; }
}
```

```
}

public class AnalyzeImageRequest
{
    public string ImageUrl { get; set; }
    public string ImageBase64 { get; set; }
}

public class AnalyzeImageResponse
{
    public string Status { get; set; }
    public string Description { get; set; }
    public string ImageFile { get; set; }
    public dynamic AnalysisResults { get; set; }
    public string Error { get; set; }
    public long ProcessingTimeMs { get; set; }
}
```

7.3.3 Program.cs - Minimal API

Setup Minimal API z czterema endpoint'ami:

```
var builder = WebApplicationBuilder.CreateBuilder(args);

// Configure services
builder.Services.Configure<VisionServiceOptions>(
    builder.Configuration.GetSection(
        VisionServiceOptions.SectionName));
builder.Services.AddScoped<IVisionService, VisionService>();
builder.Services.AddCors(opts =>
    opts.AddPolicy("AllowAll",
        policy => policy
            .AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader()));
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseCors("AllowAll");
```

```
// Endpoints
app.MapGet("/health", () => new
{
    status = "healthy",
    timestamp = DateTime.UtcNow
});

app.MapPost("/analyze-image", async (
    AnalyzeImageRequest request,
    IVisionService visionService) =>
{
    if (string.IsNullOrEmpty(request.ImageUrl) &&
        string.IsNullOrEmpty(request.ImageBase64))
        return Results.BadRequest("ImageUrl or ImageBase64 required");

    var result = string.IsNullOrEmpty(request.ImageUrl)
        ? await visionService.AnalyzeImageFromBase64Async(
            request.ImageBase64)
        : await visionService.AnalyzeImageFromUrlAsync(
            request.ImageUrl);

    return result.Status == "Success"
        ? Results.Ok(result)
        : Results.BadRequest(result);
});

app.Run();
```

7.3.4 Konfiguracja IOptions

appsettings.Development.json:

```
1 {
2   "VisionService": {
3     "Endpoint": "https://eastus.api.cognitive.
      microsoft.com",
4     "Key": "YOUR_COMPUTER_VISION_KEY"
5   }
6 }
```

7.3.5 Azure Key Vault (opcjonalnie)

Struktura gotowa do integracji:

```
var keyVaultUrl = builder.Configuration[
```

```
    "KeyVault: VaultUrl"];
if (!string.IsNullOrEmpty(keyVaultUrl))
{
    var credential = new DefaultAzureCredential();
    builder.Configuration.AddAzureKeyVault(
        new Uri(keyVaultUrl),
        credential);
}
```

7.4 API Endpoints

7.4.1 1. Health Check

```
GET /health
Response: 200 OK
{
  "status": "healthy",
  "timestamp": "2025-12-13T17:07:26.4232247Z"
}
```

7.4.2 2. Analiza obrazu (URL)

```
POST /analyze-image
Content-Type: application/json

Request:
{
  "imageUrl": "https://example.com/image.jpg"
}

Response: 200 OK
{
  "status": "Success",
  "description": "A person standing...",
  "imageFile": "https://example.com/image.jpg",
  "analysisResults": {
    "description": {...},
    "tags": [...],
    "objects": [...]
  },
  "processingTimeMs": 354,
  "error": null
}
```

7.4.3 3. Analiza obrazu (Base64)

POST /analyze-image
Content-Type: application/json

Request:

```
{  
  "imageBase64": "iVBORw0KGgoAAAANSUhEUgAAAAUA..."  
}
```

Response: jak wyzej

7.4.4 4. Test Endpoint

GET /analyze-image/test
Response: 200 OK — Analiza publicznego obrazu

7.4.5 5. Pokaż konfigurację

GET /config
Response: 200 OK

```
{  
  "visionService": {  
    "endpoint": "https://eastus.api.cognitive.microsoft.com",  
    "keyConfigured": true  
  }  
}
```

7.5 Testowanie

7.5.1 Build i uruchomienie

```
cd Net9Integration  
dotnet build  
# Output: 0 errors, 0 warnings  
dotnet run  
# Application started on http://localhost:5000
```

7.5.2 Test 1: Health Check

```
curl http://localhost:5000/health  
# Response:  
# {"status": "healthy", "timestamp": "2025-12-13T17:07:26.4232247Z"}
```

Wynik: PASSED

7.5.3 Test 2: Analiza obrazu

```
1 $body = @{  
2     imageUrl = "https://raw.githubusercontent.com/Azure -  
3     Samples/...landmark.jpg"  
4 } | ConvertTo-Json  
5 Invoke-WebRequest -Uri "http://localhost:5000/analyze-image"  
6     -Method POST '  
7     -Body $body '  
8     -ContentType "application/json"
```

Wynik: PASSED (endpoint struktura OK, wymaga rzeczywistego API key)